

BAD Design

High Level Module

High Level
Module Depends on
Low Level Module

Low Level Module

Low-Level module
controls High level
module



No Abstraction Layer

HLM: High Level Module.
LLM: Low Level Module

HLM depends on LLM

BAD Design Code

High Level Module

```
public class OrderService {
```

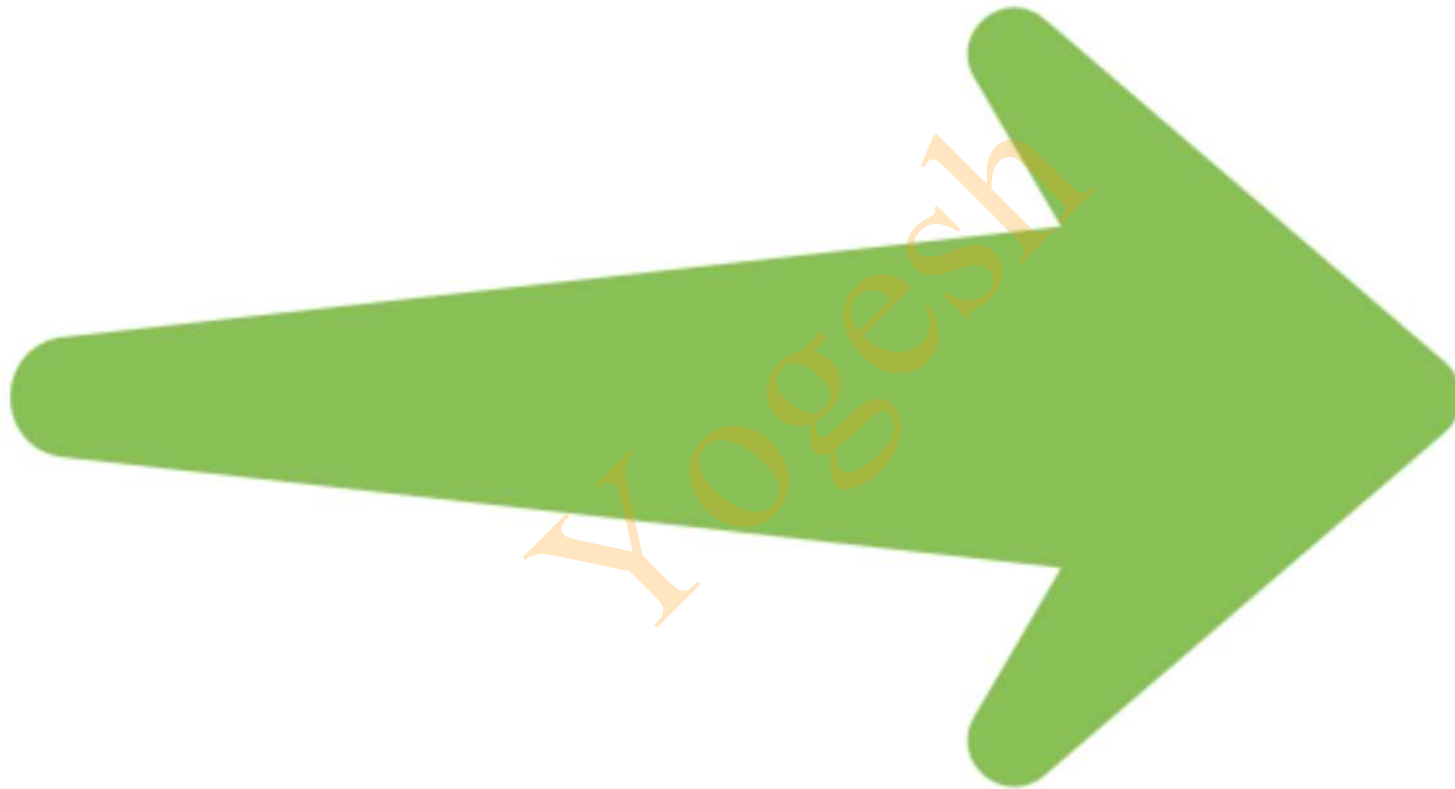
Hard Coded
Tightly Coupled

```
    private MySQLDatabase database = new MySQLDatabase();
```

```
    public void placeOrder(String orderData) {  
        database.save(orderData);  
        System.out.println("Order placed successfully");  
    }
```

```
}
```

HLM Depends on LLM



BAD Design Code

Low Level Module

```
public class MySQLDatabase {  
    public void save(String data) {  
        System.out.println("Saving to MYSQL: " + data);  
    }  
}
```

Good Design

High Level Module

"High-Level Module
Depends on
Interface (Abstraction)"

Interface Layer

Interface

Low Level Module

"Low-Level Module
Implements
Interface (Abstraction)"

Both high and low level modules
depend on abstraction,
that is, the Interface.

"Here, dependency inversion means the ownership of the contract shifts — the interface is now shaped by the High Level Module's needs, and the Low Level Module must follow it.

Both HLM and LLM still depend only on the abstraction — dependency never flows from LLM to HLM directly."

Good Design Code

High Level Module

```
public class OrderService {
```

Loosely coupled.

```
    private Database database;
```

```
    public OrderService(Database database) {  
        this.database = database;  
    }
```

```
    public void placeOrder(String orderData) {  
        database.save(orderData);  
        System.out.println("Order Placed Successfully: " + orderData);  
    }  
}
```

Good Design Code

Interface

```
public interface Database {  
    void save(String data);  
}
```

Good Design Code

Interface

```
public interface Database {  
    void save(String data);  
}
```

Low Level Module

MySQL

```
public class MySQLDatabase implements Database {  
    @Override  
    public void save(String data) {  
        System.out.println("Saving to mysql database : " + data);  
    }  
}
```



Good Design Code

Interface

```
public interface Database {  
    void save(String data);  
}
```

Low Level Module

MongoDB

```
public class MongoDBatabase implements Database {  
    @Override  
    public void save(String data) {  
        System.out.println("Saving to mongodb database : " + data);  
    }  
}
```



Let us go through the detailed explanation of definition, word by word.

Defination:

- 1.0 High-level modules should not depend on low-level modules.
Both should depend on abstractions.
- 2.0 Abstractions should not depend on details.
Details should depend on abstractions.

What do you mean by "High-Level module Depends on abstraction."?

See below how "HLM depends on Abstraction" — the first part of the definition — is proved.

```
public class OrderService {  
    private Database database;  
  
}
```

What do you mean by "low-level module depends on abstraction"?

Reamaining first part of definition:

It simply means the concrete class abides by the contract of the interface.

It has to follow all the rules of the interface.

It has to implement all the methods.

It has to follow the contract.

If it does not follow the contract, we will get a compile-time error.

Hence, it is proved that LLM depends on abstraction.

This contract is the demand of the HLM.

It means the HLM rules the LLM.

Hence, the ownership of the dependency is reversed — that is, inverted.

Please see the next slide for the code that depicts the above information.

Low Level Module

MySQL

```
public class MySQLDatabase implements Database {
```

```
    @Override
```

```
    public void save(String data) {
```

```
        System.out.println("Saving to mysql database : " + data);
```

```
    }
```

```
}
```

"MySQLDatabase implements Database" means MySQL depends on the Database interface.

What do you mean by "Abstractions should not depend on details. "?

2.0 First part meaning:-

This means the interface (abstraction) should be designed independently of any specific implementation.

The interface should not contain methods or names that belong to one particular technology.

Example: a method like `executeMySQLQuery()` is a detail — it belongs to MySQL only.

If the interface is shaped this way, every other implementation (like MongoDB) is forced to follow a contract that does not fit it.

A good interface uses generic, neutral method names — like `save()`, not `executeMySQLQuery()`.

This way, the abstraction stays independent, and any implementation can follow it cleanly.

Hence, the abstraction must stay neutral — it must not be shaped by the details of one specific implementation.

Specific Interface

// Abstraction — but shaped around ONE specific implementation (MySQL)

```
interface Database {  
    void executeMySQLQuery(String query); // MySQL-specific  
    void connectUsingJDBCdriver(String driver); // MySQL-specific  
}
```

Generic Interface

// Abstraction — generic, technology-neutral, doesn't lean toward any one implementation

```
interface Database {  
    void save(String data);  
    String fetch(String id);  
}
```

Yogesh

Since this interface is generic, any technology — like any database — can implement it.

This proves the second part of the rule: details (such as MySQL, MongoDB) should depend on the abstraction, not the other way around.